

The High-Performance Hybrid

Offloading Complex UI Without Rewriting Your App

- **Audience:** iOS & Android Native Developers
- **Core Stance:** Native-first, pragmatic offloading
- **Proof:** Live native HUD & telemetry on physical device

Image Prompt:

Minimalist dark-themed tech conference cover slide illustration. Split-screen concept: sleek Apple Swift orange bird icon and Android green robot standing together beside a glowing neon cross-platform graphics pipeline engine. 8k, modern clean vector 3D render, subtle blue-purple ambient lighting, professional presentation visual.

The Holy War

A Decade of All-or-Nothing

- **Team Native vs. Team Cross-Platform**
- **The Pain:** Rebuilding custom animations & shaders twice
- **The Result:** 2x–3x engineering effort and feature desync

Image Prompt:

Split conceptual illustration showing two frustrated mobile developers at desks: one coding Swift Metal shaders on a Mac, the other coding Android Vulkan C++ shaders on a PC, trying to match the exact same bouncing fluid animation curves on two phones. Modern tech flat illustration, subtle humorous contrast.

Precedent 1: Mobile Gaming

Why Unity Owns 70% of Top Mobile Games

- **>70% of top 1,000 games** run on Unity
- **Apple & Google provided:** SceneKit, SpriteKit, Metal, Vulkan
- **The Trade-off:** 1 unified rendering engine > 2x native rewrites

Image Prompt:

A bold tech infographic visual showing a giant glossy 3D Unity logo dominating 70% of a pie chart, surrounded by smaller logos for Apple Metal, SceneKit, and Android Vulkan. Sleek data visualization, dark background, futuristic clean glassmorphism.

Precedent 2: The Smart Shortcut

Why Meta Uses WebViews for Settings

- **Used by:** Facebook, Instagram, Amazon, Uber
- **Use Cases:** Settings, Help Centers, Terms of Service
- **The Trade-off:** High copy churn, zero 120fps physics requirement

Image Prompt:

Clean smartphone mockup showing an Instagram/Facebook Settings screen with a magnifying glass revealing clean HTML/CSS web code under the surface, seamlessly hosted within a native navigation bar. Clean vector editorial style.

The Missing Middle

When Web Is Too Weak and Native Is Too Expensive

- **Native:** High craft, but 2x–3x cost for custom graphics
- **WebView:** Low cost, but hard 30–45 fps animation ceiling
- **Add-to-App:** Native app shell + embedded high-performance engine

Image Prompt:

A clean 3-column architecture comparison diagram. Column 1: Native (Speedometer at max, but two separate stacks). Column 2: WebView (One stack, but speedometer lagging). Column 3: The Hybrid (Native shell hosting a high-fps unified graphics canvas). High-tech modern UI diagram.

The Add-to-App Myth

"Doesn't Embedding Flutter Eat 100MB RAM?"

- **Standalone Engine:** ~13–19 MB base RAM
- **FlutterEngineGroup** : ~180 KB (iOS) / ~1.4 MB (Android) per extra engine
- **Impeller Engine:** Precompiled AOT shaders, zero runtime jank

Image Prompt:

Infographic comparing memory consumption: A heavy 50MB block labeled "Old Isolated Engines" shrinking down into a sleek stack of lightweight layered cards labeled "FlutterEngineGroup ~180KB shared context". Bright glowing cyan accents, minimal clean tech chart.

The Testbed App

5 Tabs, Real Metrics, No Bullshit

- **Window-Level Native HUD:** Host `CADisplayLink` & `task_vm_info`
- **Tabs 1 & 2:** Pure Native UIKit + WKWebView
- **Tabs 3, 4, 5:** Lazily spawned engines (`/game` , `/glass` , `/scene`)

Image Prompt:

Architectural diagram of an iPhone screen with a 5-tab bar at the bottom: Tab 1 (Native Home), Tab 2 (Web Settings), Tab 3 (Flappy Cat 2D), Tab 4 (Liquid Glass), Tab 5 (3D Island). Floating transparent HUD overlay at the top showing "120 FPS | 42 MiB". Modern blueprint UI style.

Demo: Tab 1 — Pure Native

The Baseline We Never Ruin

- **Screen:** Native `HomeController` (UIKit / Compose)
- **Controls:** `UITableView`, `UISwitch`, `UISegmentedControl`
- **Verdict:** Pinned at **120 fps**; keep this 100% native!

Image Prompt:

An iPhone displaying a crisp, standard Apple iOS Settings/Dashboard table view with SF Symbols, toggles, and grouped cells. Floating green badge: "Native UIKit • 120 FPS ProMotion". High resolution mockup.

Demo: Tab 2 — WKWebView

The Ceiling of the Web

- **Normal Mode:** Smooth scrolling for text & forms
- **Stress Test:** Injected layout thrash, blur filter & JS loop
- **The Ceiling:** Framerate drops to **30–40 fps**; WebContent desync

Image Prompt:

Side-by-side comparison mockup of a mobile webview screen. Left: "Normal Mode" (clean scrolling, 60fps). Right: "Stress Test" with red warning badges, stuttering frame lines, and layout thrashing indicators. Modern UI style.

Demo: Tab 3 — Flappy Cat

120fps 2D Arcade with Flame

- **Route:** `/game` (Flame Engine on Impeller)
- **Features:** Physics, collision, particle weather, dissolve shader
- **Telemetry:** 120 fps ProMotion, UI/Raster batches ~2–4 ms

Image Prompt:

Playful pixel/vector game screen of "Flappy Cat": an orange cartoon cat flying between green obstacle pipes under a dynamic sunset sky shader, with smooth arcade particle trails. Pinned top corner HUD reading "120 FPS • 3.2ms Raster".

Demo: Tab 4 — Liquid Glass

Shaders Over the Live Native Widget Tree

- **Route:** `/glass` (GLSL Fragment Shader via Impeller)
- **Tech:** `AnimatedSampler` feeds live scrolling UI into shader
- **Features:** Real-time refraction, SDF chromatic aberration, touch ripples

Image Prompt:

High-end futuristic iOS control panel screen where a frosted translucent liquid glass card refracts and distorts a live scrolling list behind it. Glowing dynamic water ripples expanding from where a fingertip touches the glass surface. Photorealistic glassmorphism render.

Demo: Tab 5 — 3D Island Scene

`flutter_scene` + `flutter_gpu`

- **Route:** `/scene` (Low-poly 3D scene graph)
- **Features:** Inertial orbit camera, real-time Day → Night lighting
- **Centerpiece:** Tap-to-Move raycasting onto ground plane `y=0`

Image Prompt:

Beautiful low-poly 3D diorama of a tropical island floating in stylized turquoise sea: palm trees, procedural terrain, a blocky low-poly character standing on grass, with an ambient sun/moon slider. 3D Blender/Game-engine stylized isometric view.

The Bridge: Not a Walled Garden

Two-Way MethodChannel State Sync

- **Flame Game:** Invokes `reportScore` on MethodChannel
- **Native Host:** Updates `GameScoreManager` in Swift/Kotlin
- **Tab 1 Native Home:** Immediately reflects "**Highest Score**"

```
// Flutter -> Native
_gameChannel.invokeMethod('reportScore', {'score': score});
```

```
// Native iOS Swift
let score = GameScoreManager.shared.highestScore // Updated live!
```

Image Prompt:

Diagram illustrating two-way communication: A game score badge traveling across a glowing fiber-optic bridge from the Flutter Game engine directly into a native iOS UIKit table cell. High-tech clean schematic

The Cold Hard Numbers

What Did This Actually Cost Us?

- **Incremental RAM:** ~180 KB per extra engine (FlutterEngineGroup)
- **Engine Creation Time:** ~5–15 ms synchronous call duration
- **Frame Timings:** 2–5 ms UI/Raster (Budget: 8.3ms at 120Hz)
- **Engineering Velocity:** 2x–3x faster on custom cross-platform UI

Image Prompt:

Minimalist modern benchmark card showing three highlighted metric pills: "180 KB Extra RAM", "120 FPS Locked", "2.5x Dev Velocity". High contrast, clean presentation typography.

The Pragmatic Decision Matrix

Choose Tools Like an Architect

- **Keep Native (Swift/Kotlin):**

Core shell, navigation, system widgets, HealthKit/CallKit

- **Use Web (WKWebView):**

Legal disclaimers, dynamic FAQs, weekly copy changes

- **Offload to Flutter (Add-to-App):**

Mini-games, interactive canvas, shaders, 3D, promotional features

Image Prompt:

A simple 3-branch flowchart decision tree with modern clean icons: Apple/Android logo for "System Core", Web browser icon for "Static Dynamic Text", and Flutter bird icon for "High-Performance Complex Canvas".

Summary & Takeaways

Be a Pragmatic Mobile Engineer

- **1. Native is king:** Keep your native core foundation.
- **2. Respect the graphics tax:** Writing custom canvas twice costs 2x–3x.
- **3. Flutter as the "Unity for App UI":** Impeller + EngineGroup gives 120fps at ~180KB RAM.
- **4. Two-way bridge:** Seamless state flow via MethodChannel.

"Don't marry a tool. Marry shipping great software faster."

Image Prompt:

Closing slide graphic: A modern minimalist badge showing Swift, Kotlin, and Flutter logos interlocking like precision mechanical watch gears, symbolizing seamless engineering harmony. Elegant dark blue background.

Q & A

The High-Performance Hybrid

- **Codebase:** `github.com/theamorn/flutter-inapp-ios`
- **Stack:** UIKit + Compose + Flutter Impeller + Flame + `flutter_scene`
- **Slides & Docs:** `docs/presentation/`